

Technik Programista

Zaawansowane aplikacje webowe

Flask – HTML i szablony

Od stringa z HTML do Jinja2

*render_template, {{ }}, if, for, listy, słowniki, dziedziczenie szablonów,
formularze*

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. HTML w return – i dlaczego to zły pomysł.....	3
2. Szablony	3
2.1 Struktura	3
2.2 Pierwszy szablon	3
3. Zmienne i filtry	4
4. Warunki	4
5. Pętle	5
5.1 Lista	5
5.2 Słownik	5
5.3 Lista słowników – najczęstszy przypadek.....	5
7. Pliki statyczne: CSS	6
8. Formularze: GET i POST	7
8.1 GET – wyszukiwarka	7
8.2 POST – dodawanie.....	7
9. Gdzie teraz jest MVC	8
10. Zadania	9
Zadanie 1: Pierwszy szablon	9
Zadanie 2: Lista	9
Zadanie 3: Szczegóły	9
Zadanie 4: Szablon bazowy.....	9
Zadanie 5: Pusta lista	9
Zadanie 6: Szukaj (GET)	9
Zadanie 7: Dodaj (POST)	9
Zadanie 8: Repo	9

1. HTML w return – i dlaczego to zły pomysł

Najprościej: zwróć string z HTML-em.

```
@app.route("/")
def index():
    return "<h1>Sklep 4TP</h1><p>Witamy!</p>"
```

Z danymi – f-string:

```
@app.route("/produkt/<int:id>")
def produkt(id):
    nazwa = PRODUKTY.get(id, "brak")
    return f"<h1>{nazwa}</h1><p>Produkt nr {id}</p>"
```

Działą. Teraz wyobraź sobie stronę z nagłówkiem, menu, listą 50 produktów w tabeli, stopką. 200 linii HTML w stringu, w środku Pythona, z pętlą sklejącą `<tr>`. Grafik tego nie tknie. Ty za tydzień też nie.

To jest dokładnie mieszanie **widoku** z **kontrolerem**, o którym mówiliśmy przy MVC. Rozwiązanie: HTML wynosimy do osobnego pliku – **szablonu**.

2. Szablony

2.1 Struktura

```
flask-start/
├── app.py
├── templates/      <- tu szablony HTML
│   └── index.html
└── static/         <- tu CSS, obrazki, JS
```

Folder **musi** nazywać się `templates` – Flask szuka tam automatycznie (dlatego w `Flask(__name__)` podajemy `__name__`).

2.2 Pierwszy szablon

`templates/index.html`:

```
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <title>Sklep 4TP</title>
</head>
<body>
    <h1>Sklep 4TP</h1>
    <p>Witaj, {{ imie }}!</p>
</body>
</html>
```

`app.py`:

```
from flask import Flask, render_template

@app.route("/")
def index():
    return render_template("index.html", imie="Ania")
```

`render_template("plik", nazwa=wartość, ...)` wczytuje szablon, podstawia zmienne i zwraca gotowy HTML. Wszystko, co przekażesz po przecinku, jest dostępne w szablonie.

💡 *Silnik szablonów we Flasku to Jinja2. Django ma własny, ale składnia jest niemal identyczna – `{{ }}` i `{% %}`. Uczysz się raz.*

3. Zmienne i filtry

`{{ cos }}` – wstaw wartość. Może to być zmienna, atrybut, element listy, wynik prostego wyrażenia:

```
<p>{{ imie }}</p>
<p>{{ produkt.nazwa }}</p>    <!-- atrybut lub klucz słownika -->
<p>{{ produkty[0] }}</p>
<p>{{ cena * 1.23 }}</p>
<p>{{ imie|upper }}</p>        <!-- filtr -->
<p>{{ produkty|length }}</p>
<p>{{ opis|default("brak opisu") }}</p>
<p>{{ cena|round(2) }}</p>
```

Filtr to funkcja wołana przez `|`. Najczęstsze: `upper`, `lower`, `length`, `default`, `round`, `join`.

⚠️ **Jinja automatycznie „ucieka” HTML: jeśli `imie = "<b>Ania</b>"`, w przeglądarce zobaczysz dosłownie `<b>Ania</b>`, nie pogrubienie. To zabezpieczenie przed wstrzyknięciem kodu (XSS). Nie wyłączaj go.**

4. Warunki

Przekaż z kontrolera: `render_template("index.html", zalogowany=True, rola="admin")`.

```
{% if zalogowany %}
    <p>Witaj ponownie!</p>
{% else %}
    <p><a href="/login">Zaloguj się</a></p>
{% endif %}

{% if rola == "admin" %}
    <a href="/admin">Panel</a>
{% elif rola == "sprzedawca" %}
    <a href="/magazyn">Magazyn</a>
{% endif %}

{% if produkty %}          <!-- pusta lista = False -->
    ...
{% else %}
    <p>Brak produktów.</p>
{% endif %}
```

Składnia: `{% if %}` ... `{% endif %}`. Każdy blok trzeba zamknąć. Operatory jak w Pythonie: `==`, `!=`, `<`, `and`, `or`, `not`, `in`.

5. Pętle

5.1 Lista

Kontroler: `render_template("lista.html", produkty=["Laptop", "Mysz", "Klawiatura"])`

```
<ul>
{% for p in produkty %}
  <li>{{ loop.index }}. {{ p }}</li>
{% endfor %}
</ul>
```

`loop.index` – numer iteracji od 1 (`loop.index0` od 0). Jest też `loop.first`, `loop.last`, `loop.length`.

5.2 Słownik

Kontroler: `render_template("ceny.html", ceny={"Laptop": 2999, "Mysz": 49})`

```
<table>
{% for nazwa, cena in ceny.items() %}
  <tr><td>{{ nazwa }}</td><td>{{ cena }} zł</td></tr>
{% endfor %}
</table>
```

5.3 Lista słowników – najczęstszy przypadek

Tak wyglądają dane z bazy i z API. Kontroler:

```
PRODUKTY = [
    {"id": 1, "nazwa": "Laptop", "cena": 2999, "dostepny": True},
    {"id": 2, "nazwa": "Mysz", "cena": 49, "dostepny": False},
    {"id": 3, "nazwa": "Klawiatura", "cena": 199, "dostepny": True},
]

@app.route("/produkty")
def produkty():
    return render_template("produkty.html", produkty=PRODUKTY)
```

Szablon `produkty.html`:

```
<h1>Produkty ({{ produkty|length }})</h1>
<table>
  <tr><th>#</th><th>Nazwa</th><th>Cena</th><th>Status</th></tr>
  {% for p in produkty %}
    <tr>
      <td>{{ loop.index }}</td>
      <td>
        <a href="{{ url_for('produkt', id=p.id) }}">{{ p.nazwa }}</a>
      </td>
      <td>{{ p.cena }} zł</td>
      <td>{% if p.dostepny %}dostępny{% else %}<b>brak</b>{% endif %}</td>
    </tr>
  {% endfor %}
</table>
```

Zwróć uwagę na `url_for` w szablonie – działa tak samo jak w Pythonie. Linki nigdy na sztywno.

💡 *p.nazwa działa i dla słownika (p["nazwa"]), i dla obiektu (p.nazwa). Jinja sprawdza oba. Dlatego przejście ze słowników na obiekty z bazy w materiale 07 nie wymaga zmiany szablonów.*

6. Szablon bazowy

Każda strona ma ten sam `<head>`, menu i stopkę. Nie kopiuj tego do każdego pliku – zrób **szablon bazowy** z „dziurami” (**block**), które strony wypełniają.

`templates/base.html`:

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <title>{% block tytul %}Sklep 4TP{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <nav>
    <a href="{{ url_for('index') }}">Start</a> |
    <a href="{{ url_for('produkty') }}">Produkty</a>
  </nav>
  <main>
    {% block tresc %}{% endblock %}
  </main>
  <footer>4TP 2026</footer>
</body>
</html>
```

`templates/produkty.html` – teraz tylko to, co jest inne:

```
{% extends "base.html" %}

{% block tytul %}Produkty – Sklep 4TP{% endblock %}

{% block tresc %}
<h1>Produkty</h1>
<table> ... </table>
{% endblock %}
```

- `{% extends %}` – **pierwsza linia** pliku: „bazuję na base.html”.
- `{% block nazwa %}` – w bazie: miejsce do wypełnienia (z opcjonalną treścią domyślną); w stronie: wypełnienie.
- Co nie jest w bloku, w stronie potomnej jest ignorowane.

7. Pliki statyczne: CSS

Folder `static/` – Flask serwuje go pod `/static/...` Utwórz `static/style.css`:

```
body { font-family: Arial, sans-serif; max-width: 900px; margin: 2rem auto; }
nav a { margin-right: 1rem; }
table { border-collapse: collapse; }
td, th { border: 1px solid #ccc; padding: .4rem .8rem; }
```

Link w `base.html` już jest (`url_for('static', filename='style.css')`). Odśwież stronę. Jeśli CSS się nie zmienia po edycji – Ctrl + F5 (przeglądarka trzyma cache).

8. Formularze: GET i POST

8.1 GET – wyszukiwarka

Formularz GET wysyła pola jako query string. Odczyt: `request.args` (znasz z materiału 05).

```
<form method="get" action="{ url_for('szukaj') }">
  <input name="q" placeholder="Szukaj..." value="{ q or '' }">
  <button>Szukaj</button>
</form>

@app.route("/szukaj")
def szukaj():
    q = request.args.get("q", "")
    wyniki = [p for p in PRODUKTY if q.lower() in p["nazwa"].lower()]
    return render_template("szukaj.html", q=q, wyniki=wyniki)
```

8.2 POST – dodawanie

Formularz, który **zmienia dane** (dodaje, usuwa, loguje), używa POST. Dane idą w ciele żądania, nie w adresie. Odczyt: `request.form`. Jedna trasa obsługuje oba: GET pokazuje formularz, POST go przetwarza.

```
<form method="post">
  <label>Nazwa: <input name="nazwa" required></label><br>
  <label>Cena: <input name="cena" type="number" step="0.01"
required></label><br>
  <button>Dodaj</button>
</form>

@app.route("/dodaj", methods=["GET", "POST"])
def dodaj():
    if request.method == "POST":
        nowy = {
            "id": len(PRODUKTY) + 1,
            "nazwa": request.form["nazwa"],
            "cena": float(request.form["cena"]),
            "dostepny": True,
        }
        PRODUKTY.append(nowy)
        return redirect(url_for("produkty"))
    return render_template("dodaj.html")
```

- `methods=["GET", "POST"]` – bez tego POST zostanie **405 Method Not Allowed**.
- `request.form["nazwa"]` – pole z formularza; `name="nazwa"` w HTML musi się zgadzać.
- Po POST zawsze `redirect` – inaczej odświeżenie strony (F5) wyśle formularz drugi raz. Wzorec **PRG**: Post → Redirect → Get.

⚠ **Lista PRODUKTY żyje w pamięci. Restart serwera = dane znikają. To nie jest baza danych. Bazę robimy w materiale 07.**

9. Gdzie teraz jest MVC

Warstwa	U nas	Plik
Model	lista słowników <code>PRODUKTY</code> (tymczasowo)	<code>app.py</code> (góra)
View	szablony Jinja2	<code>templates/*.html</code>
Controller	funkcje z <code>@app.route</code>	<code>app.py</code>

Widok już jest osobno. Kontrolery są cienkie: biorą dane, oddają szablonowi. Zostaje model – zamiast listy w pamięci potrzebujemy czegoś, co przeżyje restart. To następny materiał.

10. Zadania

Zadanie 1: Pierwszy szablon

Przenieś stronę główną do `templates/index.html`. Przekaż z kontrolera nazwę projektu i swoje imię, wyświetl w szablonie.

Zadanie 2: Lista

Załącz w `app.py` listę 5 słowników z Twojej domeny (pola: `id`, `nazwa`, co najmniej 2 inne, jedno logiczne). Trasa `/lista` wyświetla je w tabeli z `loop.index`. Element z polem logicznym `False` ma być wyróżniony.

Zadanie 3: Szczegóły

Trasa `/element/<int:id>` wyświetla szczegóły jednego elementu w szablonie. Nieistniejące id → `abort(404)`. Nazwa w tabeli z zadania 2 ma być linkiem (`url_for`).

Zadanie 4: Szablon bazowy

`base.html` z menu (Start, Lista, Dodaj) i stopką. Wszystkie strony przez `extends`. Plik `static/style.css` podpięty.

Zadanie 5: Pusta lista

Trasa `/lista` z pustą listą pokazuje „Brak elementów”. Sprawdź `{% if %}` na pustej i pełnej liście.

Zadanie 6: Szukaj (GET)

Formularz wyszukiwania w `base.html`. Trasa `/szukaj` filtruje po nazwie, pokazuje wyniki i liczbę wyników; przy braku – komunikat.

Zadanie 7: Dodaj (POST)

Trasa `/dodaj` GET + POST. Po dodaniu przekierowanie na `/lista`. W zakładce Network pokaż POST → 302 → GET.

Zadanie 8: Repo

Commit `Flask: szablony Jinja2, base.html, formularze`. README: struktura folderów projektu.